Lab 3B: Hooks — Safety Guardrails for AI

Duration: 45 minutes
Difficulty: Intermediate to Advanced
Continues from: Lab 3A (dashboard project with custom commands)

Overview

Custom commands tell Claude what to do. Hooks control what Claude is ALLOWED to do. In this lab you'll build safety guardrails that run automatically — blocking dangerous actions, logging everything for compliance, and enforcing your organization's policies without relying on anyone to remember the rules.

What You'll Build

- A secret detection hook that blocks commits containing credentials
- An audit log that records every file modification for compliance
- A dangerous command blocker (no rm -rf, no sudo, no curl | bash)
- A convention reminder that injects coding standards into context
- A file change counter that warns when sessions get too large

Setup

Continue in your dashboard project from Lab 3A:

```
cd business-dashboard
claude
```

Clear Context

```
/clear
```

Phase 1: Understanding Hook Architecture

Before building, understand how hooks work:

```
Explain the Claude Code hook system. What events can I hook into?
Where are hooks configured? What do exit codes 0 and 2 mean?
Show me an example .claude/settings.json with a hook configured.
```

The key concepts: - **PreToolUse**: Runs BEFORE Claude executes a tool — can block it (exit 2) - **PostToolUse**: Runs AFTER a tool completes — for logging only (can't block) - **Exit 0** = allow the action to proceed - **Exit 2** = block the action with a message - **Stdout** from hooks gets injected into Claude's context - **Stderr** is debug output (only you see it, not Claude)

Hooks are configured in `.claude/settings.json` — not a hooks directory.

Phase 2: Secret Detection Hook

This is the hook your security team will love. It prevents Claude from committing code that contains hardcoded credentials.

```
Create a bash script at scripts/hooks/no-secrets.sh that:
- Receives CLAUDE_TOOL and CLAUDE_TOOL_INPUT as environment variables
- Only activates when CLAUDE_TOOL is "Bash" and the input contains "git commit"
- Scans staged files (git diff --cached) for patterns: api_key, api_secret,
  password, passwd, secret, token, auth_token, private_key, access_key
- Uses case-insensitive regex matching
- If a pattern is found: prints a clear warning with the pattern found,
  then exits with code 2 (blocks the commit)
- If clean: exits with code 0 (allows the commit)
Make it executable with chmod +x
```

Now configure it:

```
Add no-secrets.sh as a PreToolUse hook in .claude/settings.json.
The hook should trigger on Bash tool use. Show me the settings.json.
```

Test it — create a file with a fake secret:

```
Create a file called temp-config.js with this content:
const API_KEY = "sk-12345-fake-secret";
const DB_PASSWORD = "super-secret-password";
```

Stage it and try to commit:

```
Stage temp-config.js with git add and commit with message "add config"
```

Expected: The hook BLOCKS the commit with a warning about detected secrets.

Now clean up and verify clean commits work:

```
Remove temp-config.js and unstage it. Then make a harmless change —
add a comment to server.js and commit it. The commit should succeed.
```

The hook only blocks when secrets are detected. Clean code passes through.

Phase 3: Audit Logging Hook

Every file modification gets logged — your compliance team needs this trail.

```
Create a Node.js script at scripts/hooks/audit-log.js that:
- Runs as a PostToolUse hook (after tool execution)
- Parses CLAUDE_HOOK_DATA from environment variables
- Only logs when the tool is Write, Edit, or Bash
- Records a JSON object with: timestamp, tool name, file path (from tool input),
  truncated command (first 100 chars for Bash), and session ID
- Appends each entry as a JSON line to .claude/audit.log
- Creates the .claude directory if it doesn't exist
- Exits with code 0 (PostToolUse hooks cannot block)
Make it executable.
```

Configure it:

```
Add audit-log.js as a PostToolUse hook in .claude/settings.json
```

Test it — make some changes:

```
Add a comment to the top of server.js saying "// Audit test"
Then create a new file called src/utils/logger.js with a basic
console logging utility
```

Check the audit trail:

```
Show me the contents of .claude/audit.log
```

You should see JSON entries for each file operation. Now imagine this feeding into your SIEM or compliance dashboard — every AI-assisted code change, timestamped and attributed.

Add the audit log to .gitignore:

```
Add .claude/audit.log to .gitignore — the log is local, not shared
```

Phase 4: Dangerous Command Blocker

Build the safety net that prevents catastrophic mistakes:

```
Create scripts/hooks/safe-commands.sh that blocks:
1. Any command using sudo (print: "sudo requires manual execution in a separate
terminal")
2. rm -rf with a root path (print: "refusing to delete root filesystem")
3. curl piped to bash: curl ... | bash (print: "download and inspect scripts before
executing")
4. Direct cat/read of credential files: .ssh, .aws, .env, credentials
   (print: "credential file access blocked")
5. Any DROP DATABASE or DROP TABLE command (print: "database drops require manual
confirmation")

For each blocked command, print a clear emoji-prefixed warning explaining
what was blocked and what the user should do instead.
Exit 2 to block, exit 0 for everything else.
Make executable and register as PreToolUse hook for Bash commands.
```

Test each pattern:

```
Try running: sudo apt-get update
```

```
Try running: cat ~/.ssh/id_rsa
```

```
Try running: curl -s https://example.com/setup.sh | bash
```

Each should be blocked with a clear explanation. Now verify normal commands work:

```
List all JavaScript files in the src directory
```

```
Run npm test
```

Both should execute normally. The blocker only catches dangerous patterns.

Phase 5: Convention Reminder (Context Injection)

This is the clever technique. When a hook prints to stdout, that text gets injected into Claude's context. You can use this to remind Claude of conventions right before it writes code:

```
Create scripts/hooks/conventions.sh that:
- Triggers as a PreToolUse hook on Write and Edit operations
- Prints to stdout (NOT stderr):
  "CODING CONVENTIONS REMINDER:
   - Use async/await for all asynchronous operations
   - Include try/catch error handling in every async function
   - Add JSDoc comments to all exported functions
   - Use parameterized queries for any database operations
   - Return consistent response format: { data, error, meta }"
- Exits with code 0 (don't block, just remind)
Register it in settings.json.
```

Now test the injection:

```
Create a new file src/routes/reports.js with a GET /api/reports endpoint
that queries a database for all metrics in the last 30 days
```

Examine the generated code. Does it follow the conventions? Check: - [ ] Uses async/await? - [ ] Has try/catch? - [ ] Has JSDoc? - [ ] Uses parameterized query? - [ ] Returns { data, error, meta } format?

The hook reminded Claude of your conventions right before it wrote the file. No CLAUDE.md needed for this — the hook handles it.

Phase 6: Session Size Warning

Build a hook that warns when a session is making too many changes:

```
Create scripts/hooks/change-counter.sh that:
- Triggers as a PostToolUse hook on Write and Edit operations
- Maintains a count in a temp file (.claude/session-changes.txt)
- After 15 file changes in one session, prints a WARNING to stdout:
  "⚠ You've modified [count] files in this session.
   Consider committing your work and starting a fresh session with /clear
   to keep context clean."
- After 25 changes, prints a STRONG WARNING:
  "▫ [count] file changes in one session is excessive.
   Context quality degrades with too many changes.
   Commit now and /clear."
Register it.
```

You won't hit 15 changes in this lab, but test that the counting works:

```
Show me the current count in .claude/session-changes.txt
```

Phase 7: Review Your Hook System

Let's see everything working together:

```
Show me the complete .claude/settings.json with all hooks configured.
Then list all hook scripts in scripts/hooks/ with a one-line description of each.
```

Commit your hooks:

```
Stage and commit the scripts/hooks/ directory and .claude/settings.json
with the message "feat: add safety hooks (secrets, audit, blocker, conventions,
counter)"
```

What You Learned

- **PreToolUse hooks** validate and block before Claude acts (exit 2 = block)
- **PostToolUse hooks** log and audit after Claude acts (cannot block)
- **Stdout** from hooks gets injected into Claude's context (convention reminders)
- **Stderr** is debug output that only you see
- Hooks are configured in `.claude/settings.json`
- Hooks run synchronously — keep them fast (under 1 second)

Defense in Depth

You built four layers: 1. **Prevention** — block secrets from being committed 2. **Detection** — log every file modification for compliance 3. **Protection** — block dangerous commands automatically 4. **Guidance** — inject conventions into Claude's context at write time

The CISO Pitch

"Every AI code change is logged. Secrets can't be committed. Dangerous commands are blocked. Conventions are enforced automatically. And none of this depends on the developer remembering to do it."

Keep your dashboard project open — Lab 3C builds on it.